

Introduction: কেন Transformers লাইব্রেরি দরকার?

১) সমস্যাটা কী? (কেন এটা কঠিন)

Transformer মডেলগুলো সাধারণত অনেক বড়:

- মিলিয়ন থেকে শুরু করে দশ বিলিয়ন (**tens of billions**) পর্যন্ত parameters থাকতে পারে
- এত বড় মডেল **train** করা এবং **deploy** করা (প্রোডাকশনে চালানো) দুটোই কঠিন
- শুধু computation না—সাথে লাগে:
 - GPU/TPU রিসোর্স
 - মেমোরি ম্যানেজমেন্ট
 - সঠিক কনফিগারেশন
 - ইনফারেন্স অপটিমাইজেশন

আরেকটা বাস্তব সমস্যা হলো:

- প্রায় প্রতিদিনই নতুন নতুন মডেল রিলিজ হয়
- প্রত্যেক মডেলের নিজস্ব implementation, ফাইল স্ট্রাকচার, নামকরণ, লোডিং পদ্ধতি থাকতে পারে
- ফলে “সব মডেল ট্রাই করে দেখা” বা একইভাবে ব্যবহার করা খুব ঝামেলার

অর্থাৎ, তোমার হাতে যদি ১০টা মডেল থাকে, প্রত্যেকটার API আলাদা হলে তুমি:

- ১০টা আলাদা লোডিং স্টাইল শিখবে
- ১০টা আলাদা preprocessing/postprocessing পদ্ধতি সামলাবে
- ফলাফল তুলনা করা কষ্টকর হবে

এই “বিচ্ছিন্নতা/ভাঙাচোরা ইকোসিস্টেম”—টাই মূল সমস্যা।

২) Transformers লাইব্রেরি কেন তৈরি হলো?

Hugging Face Transformers লাইব্রেরি মূলত তৈরি হয়েছে এই সমস্যাটা সমাধান করতে:

যেকোনো Transformer মডেলকে
একই নিয়মে (single API)
লোড, ট্রেন, সেভ, এবং ব্যবহার করার সুবিধা দিতে

অর্থাৎ, মডেল যেই কোম্পানি/ল্যাব বানাক না কেন (Google, Meta, OpenAI-এর পুরনো ওপেন মডেল, Hugging Face community ইত্যাদি), তুমি চাইলে:

- একই ধরনের কোড লিখে মডেল লোড করতে পারবে
- tokenizer ব্যবহার করতে পারবে
- inference করতে পারবে
- model save/load করতে পারবে
- fine-tune করতে পারবে

এটাই হলো **single unified interface** বা একক API-এর মূল সুবিধা।

লাইব্রেরির মূল ফিচারগুলো (Three core features)

১) Ease of use (ব্যবহার সহজ)

লাইব্রেরির সবচেয়ে বড় শক্তি হলো:

- state-of-the-art NLP মডেল ব্যবহার করতে খুব কম কোড লাগে
- অনেক কাজ “প্রায় ready-made” চলে

তুমি `pipeline()` দিয়ে আগেই দেখেছো—দুই লাইনের মতো কোডেই inference করা যায়।
এটাই ease of use।

২) Flexibility (ফ্লেক্সিবল/কাস্টমাইজ করা যায়)

Transformers লাইব্রেরির মডেলগুলো আসলে:

- PyTorch ব্যবহার করলে: **nn.Module**
- TensorFlow ব্যবহার করলে: তার নিজস্ব model class

মানে, এগুলোকে তুমি normal ML model-এর মতোই ড্রিট করতে পারো:

- forward pass চালাতে পারো
- নিজের loss যোগ করতে পারো
- নিজের training loop লিখতে পারো
- custom layer/heads বসাতে পারো

অর্থাৎ pipeline high-level হলেও, তুমি চাইলে low-level control পাবে।

৩) Simplicity (সিম্পল/কোড বুঝতে সহজ)

Transformers লাইব্রেরির একটা বিশেষ দর্শন আছে:

“All in one file” কনসেপ্ট

মানে কী?

- একটি মডেলের forward pass সাধারণত এক জায়গায় (এক ফাইলে) লেখা থাকে
- অতিরিক্ত জটিল abstraction কম
- কোড পড়লে বোঝা যায় “ভেতরে ঠিক কী হচ্ছে”

এটা অনেক লাইব্রেরি থেকে আলাদা, কারণ অনেক সময় অন্য লাইব্রেরিতে:

- অনেকগুলো shared module থাকে
- কোড অনেক ফাইল জুড়ে ছড়িয়ে থাকে
- একটা model modify করতে গেলে বহু জায়গা touch করতে হয়

Transformers-এ সাধারণত:

- প্রতিটা মডেলের নিজস্ব layer/implementation থাকে
- তাই তুমি একটি মডেলে experiment করলে অন্য মডেলে প্রভাব পড়বে না
- debugging ও custom experiment করা সহজ হয়

এই ডিজাইন মডেলগুলোকে:

- বেশি approachable
- বেশি hackable
- বেশি শেখার উপযোগী করে

এই চ্যাপ্টারে কী শেখানো হবে?

এই introduction অংশ বলছে—এই অধ্যায়ে ধাপে ধাপে তুমি pipeline-এর ভেতরের জিনিসগুলো নিজে হাতে বুঝবে।

ধাপ ১: End-to-end example (pipeline()) কে হাতে কলমে রিপ্লিকেট)

শুরুতেই দেখাবে:

- **model + tokenizer** একসাথে কীভাবে ব্যবহার করে
- **pipeline()** function যেটা করে, সেটা নিজে হাতে কীভাবে করা যায়

মানে pipeline কীভাবে:

- text নেয়
 - tokenize করে
 - model এ দেয়
 - output decode করে
- এই পুরো flow তুমি নিজে নিয়ন্ত্রণ করবে।

ধাপ ২: **Model API** (মডেল কীভাবে লোড হয়, ভেতরে কী আছে)

এরপর আলোচনা হবে:

- Model class
- Configuration class

এখানে তুমি শিখবে:

- মডেল কীভাবে লোড হয়
- মডেল numerical inputs পেলে কীভাবে prediction দেয়
- outputs কীভাবে আসে (logits ইত্যাদি)

ধাপ ৩: **Tokenizer API (pipeline-এর অর্ধেক শক্তি)**

Tokenizer হলো pipeline-এর অত্যন্ত গুরুত্বপূর্ণ অংশ, কারণ Transformer মডেল সরাসরি text বোঝে না। Tokenizer কাজ করে দুইটা বড় জায়গায়:

(ক) **Text → Numerical inputs**

- text কে token এ ভাঙে
- token কে id তে রূপান্তর করে
- attention mask, token type ids বানায়
- model এর ইনপুট তৈরি করে

(খ) **Numerical outputs → Text**

- model থেকে output এলে
- সেটা আবার text এ ফিরিয়ে আনে (যখন দরকার)

ধাপ ৪: **Batch processing** (একসাথে অনেক বাক্য মডেলে পাঠানো)

তারপর দেখাবে:

- কিভাবে multiple sentence একসাথে batch বানিয়ে model এ পাঠাতে হয়
এটা বাস্তবে খুব দরকার, কারণ production বা training এ single sentence দিয়ে কাজ হয় না—batch লাগে।

ধাপ ৫: **High-level tokenizer() function**

শেষে তোমাকে দেখাবে:

- tokenizer() নামের high-level function কীভাবে পুরো tokenize/prepare কাজ সহজ করে দেয়
- কম কোডে ইনপুট ready করে দেয়